

HLDD AI Agent

Agentic AI Implementation (LangGraph)

Generated: May 2026

Repository: github.com/jaiminbprajapatireminder-eng/SDLC-HLDD-AI

1. What is the LangGraph Agent?

The LangGraph Agent replaces the previous single-shot chatbot with an autonomous AI agent that can reason, plan, and execute multi-step tasks using tools. Instead of following a fixed pipeline (parse -> plan -> respond), the agent uses a ReAct loop (Reasoning + Acting): it thinks about what to do next, calls a tool, gets the result, thinks again, and repeats until the task is complete.

The agent uses Groq (llama-3.1-8b-instant) as its default LLM (with Gemini as fallback), bound to 5 tools that wrap the existing HLDD application logic. The frontend provides a dedicated Agent tab (separate from the Chatbot tab) with file upload, chat messages, and JIRA controls. Human-in-the-loop is enforced via the system prompt: the agent always asks for confirmation before creating or modifying JIRA items.

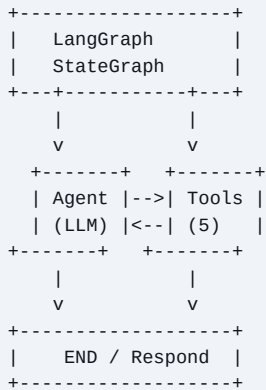
Comparison: Before vs After

BEFORE (Single-Shot Chatbot): User question -> fixed pipeline -> resp POST /api/chatbot { prompt, priority } 1. is_out_of_scope() 2. Gemini dataflow (3 stages) 3. OpenAI fallback 4. Local rule templates Returns: { response, source }	AFTER (LangGraph Agent): User goal -> agent decides -> tools -> loops -> done POST /api/agent { prompt, hldd_text, history, confirm } 1. Agent LLM (Groq llama-3.1-8b-instant + tools) 2. ToolNode (execute 1 of 5 tools) 3. Loop back to Agent LLM 4. END when task is done Returns: { response, state }
---	--

2. Agent Architecture

High-Level Component Diagram

```
POST /api/agent { prompt, hldd_text }
  |
  v
```



AGENT NODE: Groq (llama-3.1-8b-instant) + AGENT_SYSTEM_PROMPT + tool bindings

TOOL NODE: Dispatches to one of 5 registered tools

EDGES: Agent -> Tools (if tool call), Agent -> END (if response)

LOOP: Tools -> Agent (result fed back for next decision)

File Structure

```

backend/app/agent/
+-- __init__.py      -- Package marker
+-- state.py         -- AgentState TypedDict
|                   (raw_hldd_text, parsed_hldd, project_plan,
|                   epic_mapping, story_mapping, messages,
|                   user_input, pending_confirmation)
+-- prompts.py       -- AGENT_SYSTEM_PROMPT + helper templates
+-- tools.py         -- 5 LangChain @tool functions
+-- llm.py           -- Groq (llama-3.1-8b-instant) + tool binding
+-- graph.py         -- StateGraph build + conditional edges

```

3. Technology Stack

Backend

- Python 3.9+ — FastAPI framework for REST API endpoints
- LangGraph — StateGraph-based agent orchestration with ReAct loop (Agent -> Tool -> Agent)
- LangChain Groq — ChatGroq integration for llama-3.1-8b-instant as the default agent LLM
- LangChain Google GenAI — ChatGoogleGenerativeAI fallback for Gemini 2.5-flash-lite
- LangChain Core — AIMessage, HumanMessage, ToolMessage types; ToolNode for tool dispatch
- python-dotenv — environment variable loading from .env file
- PyPDF2 / python-docx — multi-format file parsing (.pdf, .docx)
- fpdf2 — PDF generation for offline documentation

Frontend

- React 18 — component-based UI with hooks (useState, useCallback, useEffect, useMemo, useRef)
- Vite — build tool and dev server
- Mermaid.js — architecture diagram rendering in the browser
- jsPDF + html2canvas — PDF export from DOM
- xlsx — Excel (.xlsx) export for Features, Stories, Testing tabs

Agentic AI Architecture

The LangGraph agent lives in backend/app/agent/ and follows a StateGraph pattern:

```
backend/app/agent/
+-- __init__.py      -- Package marker
+-- state.py         -- AgentState: messages, raw_hldd_text, parsed_hldd,
|                     project_plan, epic_mapping, story_mapping,
|                     user_input, pending_confirmation
+-- llm.py           -- LLM builder: Groq (llama-3.1-8b-instant) primary,
|                     Gemini (gemini-2.5-flash-lite) fallback
+-- tools.py         -- 5 @tool functions wrapping app logic
+-- prompts.py       -- AGENT_SYSTEM_PROMPT (concise, ~800 chars)
+-- graph.py         -- StateGraph: agent_node -> ToolNode -> process_tool_results
```

LangGraph StateGraph Structure

```
+-----+
|  START  |
+-----+
|
v
+-----+
|  AGENT  | (Groq LLM + tool binding)
+-----+
|
+-- tool_calls? --Yes--> +-----+
|                        |  TOOLS  | (ToolNode: dispatch to matched tool)
| No                    +-----+
|                        |
v                        v
+-----+                +-----+
|  END   |                | process_results | (extract parsed/plan from tool output)
+-----+                +-----+
|
v
(back to AGENT)
```

```
Decision function: should_continue()
- Last message has tool_calls? -> go to TOOLS
- No tool_calls? -> go to END
```

Token Optimization for Groq Free Tier

The Groq free tier has a strict 6000 tokens-per-minute (TPM) limit per request. Several optimizations were applied to stay within this limit:

- HLDD text truncated to 1500 characters before sending to the LLM
- Conversation history limited to last 6 messages to control cumulative token growth

- AI response content truncated to 500 characters per message
- Tool result content truncated to 600 characters per message
- Combined `parse_and_generate_plan` tool replaces two separate tools (`parse_hldd` + `generate_plan`)
- System prompt condensed from ~1900 to ~800 characters
- Reduced from 7 to 5 tools, cutting tool schema overhead by ~30%
- Graceful 429 HTTP response when rate limit is hit, with upgrade suggestion

GROQ_API_KEY Configuration

The `GROQ_API_KEY` environment variable is the PRIMARY authentication method for the Agentic AI agent. When set, the LangGraph agent uses Groq (llama-3.1-8b-instant) as its reasoning engine. This provides faster inference and more reliable tool calling compared to Gemini. Users with paid Groq tiers can set `GROQ_MODEL=llama-3.3-70b-versatile` for higher quality output.

- Default model: llama-3.1-8b-instant (30,000 TPD free tier limit)
- Paid upgrade: llama-3.3-70b-versatile (set `GROQ_MODEL` env var)
- Fallback: `GEMINI_API_KEY` is used when `GROQ_API_KEY` is not set
- Rate limits: ~6000 TPM on free tier — optimized via prompt truncation

LLM Fallback Chain

Agent LLM Priority:

1. `GROQ_API_KEY` set -> Groq llama-3.1-8b-instant (default)
2. `GEMINI_API_KEY` set -> Gemini gemini-2.5-flash-lite (fallback)
3. Neither set -> `ValueError` (at least one required)

Override: `GROQ_MODEL` env var for paid tiers (e.g. llama-3.3-70b-versatile)

4. Agent State Schema

```
AgentState = {  
    raw_hldd_text: str          # Original HLDD document text  
    parsed_hldd: dict | None    # Output of parse_hldd tool  
    project_plan: dict | None    # Output of generate_plan tool  
    epic_mapping: dict          # feature_id -> JIRA epic key  
    story_mapping: dict          # story_id -> JIRA story key  
    messages: list              # Chat history (Human/AI/Tool msgs)  
    user_input: str             # Latest prompt from user  
    pending_confirmation: str    # Pending action awaiting user OK  
}
```

5. Tool Definitions

`parse_and_generate_plan(raw_hldd_text: str) -> dict`

- Wraps `parser.parse_hldd_document()` then `generator.generate_project_plan()` in one step. Extracts title, functional_requirements, technology_stack, and immediately generates features, stories, testing stories, architecture diagram, and project structure. Preferred over calling `parse_hldd` + `generate_plan` separately, especially for smaller LLMs.
- Wraps `gemini_dataflow.call_gemini_dataflow()`. For general Q&A, technology comparisons, cost analysis, architecture advice, risk assessment. Always passes current HLDD context.
- Creates an Epic in JIRA SCRUM project. Returns JIRA key (e.g., SCRUM-42). Must be called BEFORE `create_jira_story`.
- Creates a Story in JIRA linked to a parent Epic via the parent field. Requires `epic_key` from `create_jira_epic`.
- Creates a Subtask in JIRA linked to a parent Story. Requires `parent_key` from `create_jira_story`.

6. Prompts Used

5.1 AGENT_SYSTEM_PROMPT (master prompt)

This is the most important prompt -- it defines the agent's personality, capabilities, workflow rules, and human-in-the-loop requirements. It lives in prompts.py.

```
"""You are an HLDD Delivery Agent that analyzes High-Level Design
Documents and manages JIRA delivery.

## Tools
- parse_and_generate_plan: Parse HLDD and generate features/stories/tests.
- ask_gemini: Ask Gemini about the project, tech comparisons, or recommendations.
- create_jira_epic: Create a JIRA Epic for a feature (ask user first).
- create_jira_story: Create a JIRA Story linked to an Epic (ask user first).
- create_jira_subtask: Create a JIRA Subtask linked to a Story (ask user first).

## Rules
1. When user provides HLDD text, call parse_and_generate_plan. Report results.
2. For JIRA: only create when user explicitly asks. Always ask confirmation first.
3. For questions: use ask_gemini.
4. Be concise. If no HLDD provided, ask the user to provide one.
"""
```

5.2 Tool Docstrings (per-tool prompts)

Each @tool function's docstring IS a prompt -- it tells the LLM when to call the tool, what inputs it needs, and what it returns. The LLM reads these docstrings at inference time via Gemini's function calling / tool binding.

5.3 Context Injection (dynamic prompt)

Before each agent turn, the agent_node function injects the current HLDD context into the system message. This is built dynamically from the state:

```
## Current HLDD Context
Title: {title}
Tech Stack: React, FastAPI, PostgreSQL
Functional Requirements: 4

## Current Project Plan
Features: 4
Development Stories: 12
Testing Stories: 12
JIRA Epics Created: 2 (SCRUM-50, SCRUM-51)
JIRA Stories Created: 0
```

7. Agent Execution Flow

Example: User says 'Parse this HLDD and create Epics in JIRA'

```

Step 1: Agent LLM receives system prompt + user message
  Thinks: 'I need to parse the HLDD and generate a plan'
  Calls: parse_and_generate_plan(raw_text)
  State: parsed_hldd = { title, tech_stack, ... }
         project_plan = { features: [...], stories: [...] }

Step 2: process_tool_results extracts parsed_hldd and project_plan
  from the ToolMessage JSON output, then loops back to Agent

Step 3: Agent LLM sees plan. User asked for JIRA.
  Thinks: '4 features found. Must ask user before creating.'
  Responds: 'I found 4 features. Shall I create 4 Epics in JIRA?'
  Pauses (waits for user input)

Step 4: User says 'Yes, proceed'
  Agent: Calls create_jira_epic() x 4
  Results: SCRUM-50, SCRUM-51, SCRUM-52, SCRUM-53
  State: epic_mapping = { IPMA#001: SCRUM-50, ... }

Step 5: Agent reports results
  Responds: 'Created 4 Epics: SCRUM-50 through SCRUM-53'
  END

```

Graph Transitions

```

Agent Node -> should_continue()
|
+-- Last msg has tool_calls? --> Tools Node
|
|                               |
|                               +-- Execute tool
|                               +-- Return result as ToolMessage
|                               +-- Go back to Agent Node
|
+-- Last msg has NO tool_calls? --> END
|
+-- Return final response

```

8. Human-in-the-Loop Design

- The system prompt instructs the agent to ALWAYS ask before JIRA actions
- The LLM generates the confirmation question naturally (no code-level interrupt)
- User responds with 'yes'/'proceed'/'confirm' -- agent then calls JIRA tools
- If user says 'no'/'cancel'/'stop' -- agent does NOT call JIRA tools and asks what next
- The confirm field in the AgentRequest allows frontend to pass explicit approval
- All 6 tools are pure functions -- they cannot modify state without being called

9. FastAPI Integration

Endpoint: POST /api/agent

```
Request:
{
  'prompt': 'Parse this HLDD and create Epics for all features',
  'hldd_text': 'HLDD document content...',
  'history': [
    { 'role': 'user', 'content': '...' },
    { 'role': 'assistant', 'content': '...' }
  ],
  'confirm': false          # set true to confirm pending action
}

Response:
{
  'response': 'Created 4 Epics: SCRUM-50, SCRUM-51...',
  'state': {
    'parsed': { ... },      # parsed HLDD data
    'plan': { ... },        # project plan
    'epics': { IPMA#001: SCRUM-50, ... },
    'stories': {}
  }
}
```

Error Handling

- If GEMINI_API_KEY is not set: endpoint returns HTTP 500 with clear error
- If JIRA_EMAIL/JIRA_API_TOKEN not set: JIRA tools return error string (not crash)
- Tool failures are returned as ToolMessage content, fed back to agent LLM
- Agent LLM reads the error and asks the user how to proceed

10. E

Variable	Required	Description

GROQ_API_KEY	Preferred	Groq key for agent LLM (llama-3.1-8b-instant)
GEMINI_API_KEY	Fallback	Gemini API key (fallback when GROQ_API_KEY not set)
JIRA_EMAIL	For JIRA	JIRA Cloud email
JIRA_API_TOKEN	For JIRA	JIRA Cloud API token
OPENAI_API_KEY	No	OpenAI key (for chatbot fallback)
GROQ_MODEL	No	Override default Groq model

11. Before vs After: What Changed

Architecture

- ✗ Before: Single-shot chatbot endpoint
- ✓ After: LangGraph StateGraph with ReAct loop

LLM Integration

- ✗ Before: 3-stage Gemini dataflow + OpenAI + local rules
- ✓ After: Groq llama-3.1-8b-instant with 5 tools

Response Generation

- ✗ Before: Fixed pipeline (analyze -> reason -> generate)
- ✓ After: Agent decides: think -> tool -> think -> tool -> done

JIRA Sync

- ✗ Before: 3 separate manual UI buttons
- ✓ After: Agent calls JIRA tools autonomously (with confirmation)

Human-in-the-Loop

- ✗ Before: None
- ✓ After: Agent asks user before any JIRA action

State Management

- ✗ Before: LATEST_PAYLOAD in-memory dict
- ✓ After: AgentState with full conversation + mappings

Context Handling

- ✗ Before: build_chat_context() stuffed into every prompt
- ✓ After: Dynamic context injection from AgentState

Extensibility

- ✗ Before: Adding new features requires new code paths
- ✓ After: Adding a new @tool function instantly gives agent new capability

12. File Upload via Agent Chat

The frontend provides a dedicated Agent tab (separate from the Chatbot tab) with a file upload card and a chat interface. Users can upload .txt, .md, .docx, or .pdf HLDD documents directly to the agent without using the main /api/upload endpoint.

Upload Flow

1. User clicks file input in agent chat upload card
2. Frontend reads file as text (FileReader / .text())
3. Frontend calls POST /api/agent with:


```
{ prompt: 'Parse this HLDD...', hldd_text: '<file content>', history: [] }
```
4. Agent LLM receives file content as hldd_text
5. Agent decides to call parse_and_generate_plan(raw_text) tool
6. Returns structured sections + project plan in one call
7. Agent reports results to user in chat
8. User can continue: 'Create Epics in JIRA', 'Summarize the plan', etc.

Example Prompts for the Agent

Prompt	Agent Action
'Upload an HLDD file and parse it'	Prompts user to upload
'What can you do?'	Lists capabilities
'Summarize the delivery plan'	Summarizes parsed plan
'Create Epics in JIRA for all features'	Creates Epics (asks first)
'Create Stories linked to the Epics'	Creates Stories
'Create Subtasks for all Stories'	Creates Subtasks
'Compare AWS vs GCP for this project'	Calls ask_gemini tool

'What are the risks?'	Risk analysis via Gemini
'Yes' / 'Proceed'	Confirms pending JIRA action
'No' / 'Cancel'	Cancels pending action

13. Multi-Format File Upload

The frontend agent chat supports uploading .txt, .md, .docx, and .pdf files directly to the agent. For binary formats (.docx, .pdf), the frontend first uploads the file to POST /api/extract-text which reuses the existing _read_upload() logic (supporting python-docx and PyPDF2), then passes the extracted text to the agent as hldd_text.

Upload Flow for Binary Files

1. User selects .pdf or .docx file in agent chat
 2. Frontend calls POST /api/extract-text (FormData with file)
 3. Backend calls _read_upload() -> returns extracted text
 4. Frontend receives { text, filename }
 5. Frontend calls POST /api/agent with hldd_text=text
 6. Agent parses the text and generates a project plan

Supported Formats

Format	Method

.txt	file.text() in browser (client-side)
.md	file.text() in browser (client-side)
.docx	POST /api/extract-text (python-docx)
.pdf	POST /api/extract-text (PyPDF2)

14. Future Enhancements

- Add `interrupt_before=['tools']` in `graph.compile()` for code-level human-in-the-loop (pauses before every tool)
- Add RAG tool that retrieves relevant HLDD chunks via vector search for large documents
- Add `web_search` tool for real-time technology/cost research
- Add MCP integration for IDE tool access (file editing, git operations)
- Add streaming response via Server-Sent Events so the agent's reasoning is visible in real-time
- Add persistent state via SQLite checkpointing (LangGraph's built-in `MemorySaver`)
- Replace Gemini with a fine-tuned model specialized in HLDD analysis